

Lab Exercise 1: Triggering Nested Loop Join

Query

Query History

```
1 EXPLAIN (ANALYZE , BUFFERS )
2 SELECT c . first_name , c . last_name , r . rental_date
3 FROM customer c
4 JOIN rental r ON c . customer_id = r . customer_id
5 WHERE c . customer_id = 1 ;
```

Data Output

Messages

Notifications

≡

📄

▼

📄

▼

🗑

📄

📥

📈

SQL

Showing rows: 1 to 1

QUERY PLAN

text

🔒

1	Nested Loop (cost=0.28..359.16 rows=32 width=21) (actual time=1.282..5.153 rows=32.00 loops=1)
2	Buffers: shared hit=1 read=152
3	-> Index Scan using customer_pkey on customer c (cost=0.28..8.29 rows=1 width=17) (actual time=0.082..0.085 rows=1.00 loops=1)
4	Index Cond: (customer_id = 1)
5	Index Searches: 1
6	Buffers: shared hit=1 read=2
7	-> Seq Scan on rental r (cost=0.00..350.55 rows=32 width=10) (actual time=1.196..5.056 rows=32.00 loops=1)
8	Filter: (customer_id = 1)
9	Rows Removed by Filter: 16012
10	Buffers: shared read=150
11	Planning:
12	Buffers: shared hit=148 read=4 dirtied=1
13	Planning Time: 7.169 ms
14	Execution Time: 5.184 ms

- customer c is the Outer (top) and rental r is the Inner (bottom)
- The customer table returned only one row. Therefore, the Inner table only needed to run once, so building a Hash Table would add unnecessary overhead.
- because Outer tables return only 1 row, the Inner Table is executed only once

Lab Exercise 2: Triggering Hash Join

```
1  EXPLAIN (ANALYZE , BUFFERS )
2  SELECT f.title , l.name as language
3  FROM film f
4  JOIN language l ON f . language_id = l.language_id ;
5  -- CREATE INDEX idx_fk_rental_customer_id ON rental(customer_id);
```

Data Output Messages Notifications

QUERY PLAN

text

1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=1.794..2.038 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=88 read=1
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.020..0.145 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=1.732..1.733 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=1.716..1.718 rows=6.00 loops=1)
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=23 read=1
13	Planning Time: 1.709 ms
14	Execution Time: 2.092 ms

- language l is being Hashed (Built). film f is being Probed?
- 9 kB
- Memory Usage còn 1kB

Lab Exercise 3: Triggering Merge Join

```
1 SET enable_hashjoin = off;
2 SET enable_nestloop = off;
3 EXPLAIN (ANALYZE , BUFFERS )
4 SELECT f.film_id , f.title , i.inventory_id
5 FROM film f
6 JOIN inventory i ON f.film_id = i.film_id
7 ORDER BY f.film_id ;
```

Data Output Messages Notifications

Showing rows: 1 to

	QUERY PLAN	
	text	
1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=2.743..4.258 rows=4581.00 loops=1)	
2	Merge Cond: (f.film_id = i.film_id)	
3	Buffers: shared hit=85 read=25, temp read=10 written=12	
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.007..0.216 rows=1000.00 loo...	
5	Index Searches: 1	
6	Buffers: shared hit=84 read=1	
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=2.730..3.404 rows=4581.00 loops=1)	
8	Storage: Memory Maximum Storage: 17kB	
9	Buffers: shared hit=1 read=24, temp read=10 written=12	
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=2.722..3.036 rows=4581.00 loops=1)	
11	Sort Key: i.film_id	
12	Sort Method: external merge Disk: 80kB	
13	Buffers: shared hit=1 read=24, temp read=10 written=12	
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.029..0.740 rows=4581.00 loops=1)	
15	Buffers: shared hit=1 read=24	
16	Planning:	
17	Buffers: shared hit=5 read=1	
18	Planning Time: 0.851 ms	
19	Execution Time: 5.573 ms	

- Both. The film table used an Index Scan to provide sorted data, while the **inventory** table required a Sort operation
- Yes. PostgreSQL switched back to Hash Join because both tables were joined without a restrictive filter, and the equality condition made Hash Join cheaper.

1	EXPLAIN (ANALYZE , BUFFERS)
2	SELECT f.film_id , f.title , i.inventory_id
3	FROM film f
4	JOIN inventory i ON f.film_id = i.film_id
5	ORDER BY f.film_id ;
Data Output Messages Notifications	
Showing	
	QUERY PLAN
	text
1	Sort (cost=471.95..483.40 rows=4581 width=23) (actual time=2.884..3.076 rows=4581.00 loops=1)
2	Sort Key: f.film_id
3	Sort Method: quicksort Memory: 383kB
4	Buffers: shared hit=116
5	-> Hash Join (cost=110.50..193.39 rows=4581 width=23) (actual time=0.761..2.021 rows=4581.00 loops=1)
6	Hash Cond: (i.film_id = f.film_id)
7	Buffers: shared hit=113
8	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.036..0.407 rows=4581.00 loops=1)
9	Buffers: shared hit=25
10	-> Hash (cost=98.00..98.00 rows=1000 width=19) (actual time=0.713..0.714 rows=1000.00 loops=1)
11	Buckets: 1024 Batches: 1 Memory Usage: 59kB
12	Buffers: shared hit=88
13	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=19) (actual time=0.018..0.441 rows=1000.00 loops=1)
14	Buffers: shared hit=88
15	Planning:
16	Buffers: shared hit=330
17	Planning Time: 6.626 ms
18	Execution Time: 3.454 ms

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

Query	Query History
1	SET enable_indexscan = OFF;
2	
3	EXPLAIN ANALYZE
4	SELECT *
5	FROM rental
6	WHERE rental_date > '2005-05-24';
Data Output Messages Notifications	
Showing	
	QUERY PLAN
	text
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.051..3.067 rows=16044.00 loops=1)
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)
3	Buffers: shared hit=150
4	Planning:
5	Buffers: shared hit=3
6	Planning Time: 0.460 ms
7	Execution Time: 4.144 ms

Query

Query History

```

1  SET enable_indexscan = ON;
2
3  EXPLAIN ANALYZE
4  SELECT *
5  FROM rental
6  WHERE rental_date > '2005-05-24';

```

Data Output

Messages

Notifications

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

QUERY PLAN

text

🔒

1

Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.023..2.437 rows=16044.00 loop...

2

Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)

3

Buffers: shared hit=150

4

Planning:

5

Buffers: shared hit=3

6

Planning Time: 0.223 ms

7

Execution Time: 3.342 ms

```

1  RESET ALL;
2
3  EXPLAIN (ANALYZE, BUFFERS)
4  SELECT
5      c.customer_id,
6      c.first_name,
7      c.last_name,
8      r.rental_id,
9      r.rental_date,
10     i.inventory_id
11  FROM customer c
12  JOIN rental r ON c.customer_id = r.customer_id
13  JOIN inventory i ON r.inventory_id = i.inventory_id;

```

Data Output

Messages

Notifications

≡

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows

	QUERY PLAN text 🔒
1	Hash Join (cost=150.55..545.57 rows=16044 width=33) (actual time=6.603..14.761 rows=16044.00 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=177 read=7
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=33) (actual time=5.419..10.185 rows=16044.00 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=152 read=7
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=18) (actual time=0.029..0.878 rows=16044.00 loop...)
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=5.372..5.374 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 38kB

Query Query History

```
1 ANALYZE customer;
2 ANALYZE rental;
3 ANALYZE inventory;
```

Data Output Messages Notifications

ANALYZE

Query returned successfully in 126 msec.

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT
3     c.customer_id,
4     c.first_name,
5     c.last_name,
6     r.rental_id,
7     r.rental_date,
8     i.inventory_id
9 FROM customer c
10 JOIN rental r ON c.customer_id = r.customer_id
11 JOIN inventory i ON r.inventory_id = i.inventory_id;
```

Data Output Messages Notifications

≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL Showing ro	
	QUERY PLAN text 🔒
1	Hash Join (cost=150.55..545.57 rows=16044 width=33) (actual time=3.496..17.335 rows=16044.00 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=184
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=33) (actual time=0.843..8.825 rows=16044.00 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=159
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=18) (actual time=0.066..1.504 rows=16044.00 loop...
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.742..0.744 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 38kB
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=4) (actual time=0.055..1.002 rows=4581.00 loops=1)
12	Buffers: shared hit=25
13	Planning:
14	Buffers: shared hit=64
15	Planning Time: 1.250 ms
16	Execution Time: 18.381 ms